Applied Programming

Week 2: Functions, Tests, and NumPy Review

Path Geometry and Verification

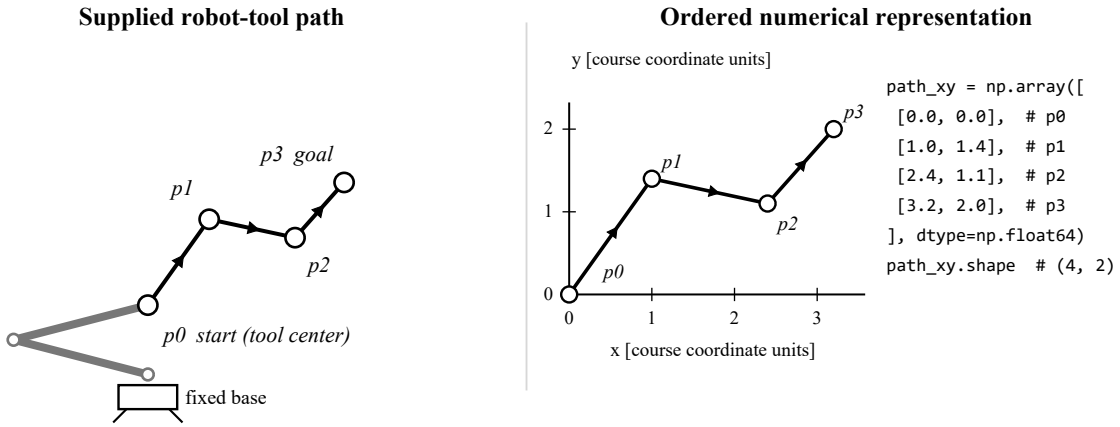

Figure 2.1. The same supplied robot-tool path has a physical interpretation and an ordered numerical representation.

The left side of Figure 2.1 gives the four waypoints a physical interpretation: they are successive tool-center positions along one supplied robot-tool path. The right side records the same ordered positions as an x-y coordinate plot and a NumPy-ready array. Row i stores waypoint pi as (x,y); changing row order changes the path. A display can still appear plausible when code skips an endpoint, exchanges coordinates, or calculates the wrong length, so the programming question is how to publish this familiar calculation as a reusable function and support it with numerical evidence.

The chapter develops four functions in dependency order. The first calculates one segment length, the second samples one segment, the third calculates one polyline length, and the fourth applies the same meaning to a batch of paths. Python containers make the data flow visible first; NumPy then makes shapes and axes explicit without changing that meaning.

Function	Result that must be verified
`segment_length(...)`	one nonnegative distance
`interpolate_segment(...)`	exactly M points, including both endpoints
`path_length(...)`	sum of consecutive segment lengths
`path_lengths(...)`	one length for every path in a batch

The final evidence is not one screenshot. It consists of 48 published tests, at least six independent student tests, numerical output, an animated SVG generated by provided display code, a built wheel, and one identifiable Git revision submitted through LMS.

2.1 Python review: values become evidence only when their data flow is clear

A program first creates values and gives them names. Assignment binds a name to the result of an expression; it does not write an equation that remains true forever. A function call is also an expression. Its arguments are bound to parameters, the body executes, and a return statement supplies the value of the call to the surrounding program.

```
start = (0.0, 0.0)
goal = (3.0, 4.0)

dx = goal[0] - start[0]
dy = goal[1] - start[1]
squared_length = dx * dx + dy * dy

def finish_distance(value: float) -> float:
    return value**0.5

distance = finish_distance(squared_length)
print(distance)
```

```
5.0
```

The names `dx` and `dy` retain the two coordinate differences. The expression inside `finish_distance` creates a numerical value, and `return` makes that value available to the caller. By contrast, `print` writes text to the terminal. A test can compare a returned number directly; it should not have to recover the number from display text.

Construct	Meaning in this program
name	a label bound to a Python object
assignment	evaluate the right side, then bind the result
expression	produce a value
function call	bind arguments, execute the body, and produce the returned value
`print(...)`	display text as a side effect; it is not a numerical return value

This small 3-4-5 calculation is intentionally easier to check by hand than the four-waypoint path in Figure 2.1. It lets the chapter review familiar Python constructs while every intermediate value remains visible. After the scalar function and its tests are established, the chapter returns to the supplied four-waypoint path and then extends the same meaning to NumPy batches.

A tuple records one point; a list records the ordered path

A two-dimensional tool-center point is an ordered pair. A tuple can store one point, while a list can collect the ordered waypoints of a path. Python does not know that the first slot means x and the second means y; that interpretation belongs to the course contract.

```
start = (0.0, 0.0)
corner = (3.0, 0.0)
goal = (3.0, 4.0)
waypoints = [start, corner]
waypoints.append(goal)

x_goal, y_goal = goal
first_two = waypoints[:2]
print(waypoints[1], first_two, x_goal, y_goal)
```

```
(3.0, 0.0) [(0.0, 0.0), (3.0, 0.0)] 3.0 4.0
```

Indexing selects one item, so `waypoints[1]` is the second waypoint. Slicing selects a range and excludes the stop index, so `[:2]` contains indices 0 and 1. Unpacking binds the two tuple positions to two names in the same order. `append` changes the list by adding one waypoint at its end.

Python operation	Path interpretation
`point = (x, y)`	one ordered tool-center point
`waypoints = [...]`	one ordered waypoint sequence
`waypoints[i]`	waypoint i
`waypoints[a:b]`	path prefix or subpath from a through b-1
`x, y = point`	unpack two positions without changing their order
`waypoints.append(point)`	extend the sequence by one waypoint

A tuple is not automatically a point and a list is not automatically a path. Their meaning is established by the declared coordinate order and by preserving list order throughout the program.

Iteration exposes the N waypoints to N - 1 segment relation

A path becomes computational when the same rule is applied to each consecutive pair. The two slices below have equal length and differ by one starting position. `zip` therefore yields `(p0,p1)`, then `(p1,p2)`. The result has one fewer segment than waypoints.

```
segments = []
for p0, p1 in zip(waypoints[:-1], waypoints[1:]):
    displacement = (p1[0] - p0[0], p1[1] - p0[1])
    segments.append(displacement)

for index, displacement in enumerate(segments):
    print(index, displacement)

segments_comp = [
    (p1[0] - p0[0], p1[1] - p0[1])
    for p0, p1 in zip(waypoints[:-1], waypoints[1:])
]
```

```
0 (3.0, 0.0)
1 (0.0, 4.0)
```

The explicit loop and the list comprehension express the same ordered calculation. The explicit form is easier to trace while reviewing familiar Python; the comprehension is useful only after the input order and output meaning are stable. `enumerate` adds an index without changing the stored displacement.

```
import numpy as np

if len(waypoints) < 2:
    raise ValueError("a path needs at least two waypoints")

path = np.asarray(waypoints, dtype=np.float64)
print(path.shape)
```

```
(3, 2)
```

The `if` statement detects a declared invalid condition and `raise` stops the call with an exception. `np.asarray` then converts the valid ordered values into a homogeneous numerical array. The shape `(3,2)` records three waypoints and two coordinates per waypoint.

2.2 A reusable function publishes more than a formula

A calculation inside one script may produce the expected number, yet a caller still needs to know what may be passed, what is returned, which units are used, which boundaries are valid, and how invalid input is reported. Together these statements form the public function contract.

```
from collections.abc import Sequence

import numpy as np
from numpy.typing import NDArray


PointLike = Sequence[float] | NDArray[np.integer | np.floating]


def segment_length(
    start_xy: PointLike,
    goal_xy: PointLike,
) -> float:
    """Return Euclidean segment length in course coordinate units.

    Both inputs contain finite real values ordered as (x, y).
    Raise PathInputError when an input violates the public contract.
    """
    ...
```

Contract part	Published meaning
input	two points normalized to shape `(2,)`
output	one finite, nonnegative Python float
unit	course coordinate units
valid boundary	identical endpoints are accepted and return 0
invalid input	raise `PathInputError`; do not clip or return a sentinel
side effects	do not print, plot, write files, or mutate caller-owned data

The signature names the callable boundary. The type hints and docstring communicate the intended values to readers and tools. They do not themselves prevent a string, a three-coordinate tuple, a Boolean, or a `NaN` from reaching the function at runtime.

Type hints describe intent; validation checks runtime input

Type hints are not automatic runtime guards. Python ordinarily accepts a call first and executes the function body. A public engineering function must therefore check the actual object at the boundary before applying a numerical relation.

Mechanism	What it contributes	What it does not guarantee
signature	parameter names, order, defaults, return position	engineering validity
type hint	intended types for readers and static tools	automatic rejection at runtime
docstring	units, shapes, boundary, failure behavior	executable verification
runtime validation	checks actual values before computation	correct numerical formula
test	evidence for selected contract statements	proof for every possible input

```
class PathInputError(ValueError):
    """Raised when public path data violates the Week 2 contract."""


def segment_length(start_xy, goal_xy) -> float:
    start = _as_point(start_xy, name="start_xy")
    goal = _as_point(goal_xy, name="goal_xy")
    delta = goal - start
    return float(np.sqrt(np.sum(delta * delta)))
```

The public exception communicates that the failure belongs to the path-data contract rather than to an unrelated file or plotting operation. The helper normalizes both inputs before the Euclidean calculation. The function returns a Python `float` so that the public interface does not expose a NumPy scalar accidentally.

Identical endpoints are a valid boundary, not invalid data. Their displacement is zero and the returned length is zero. This distinction must appear both in the written contract and in the tests.

Validation checks representation, shape, numeric type, and finiteness

Different invalid values fail for different reasons. A string or Boolean is not an accepted coordinate representation. A three-entry value has the wrong shape. `NaN` and infinity are numerical values but are not finite coordinates. A NumPy array whose `dtype` is `object` is also rejected because it does not declare one homogeneous numerical representation. The helper applies these checks in a stable order so that every public function reports the same boundary.

```
def _as_point(value, *, name: str) -> np.ndarray:
    if isinstance(value, np.ndarray) and value.dtype == object:
        raise PathInputError(f"{name} must use a numerical dtype")

    raw = np.asarray(value, dtype=object)
    if raw.shape != (2,):
        raise PathInputError(f"{name} must have shape (2,)")

    number_types = (int, float, np.integer, np.floating)
    if any(
        isinstance(item, (bool, np.bool_))
        or not isinstance(item, number_types)
        for item in raw.flat
    ):
        raise PathInputError(f"{name} must contain real coordinates")

    array = raw.astype(np.float64)
    if not np.all(np.isfinite(array)):
        raise PathInputError(f"{name} must contain finite values")
    return array
```

The leading underscore marks `_as_point` as an internal helper, not as one of the four functions students publish. Boolean is rejected explicitly because Python treats `bool` as related to integer, while the course contract does not interpret `True` as a coordinate. The helper does not reshape arbitrary input or replace invalid values, because silent repair would change the caller's data without an agreed engineering rule.

> Boundary rule. Normalize only representations allowed by the contract. Reject wrong shape, unsupported value type, and non-finite data before numerical computation.

A module boundary separates numerical functions from external tools

The numerical module should answer numerical questions only. Printing, creating an SVG, building a wheel, and invoking an external tool are separate responsibilities. Keeping those side effects outside the numerical functions lets tests, later planners, command-line scripts, and visual demonstrations reuse the same returned arrays.

File or component	Responsibility	Student changes
`path_geometry.py`	validation and the four numerical functions	yes
student tests and note	independent tests and interpretation of evidence	yes
protected demo, visualizer, scripts	call, display, and verify public functions	no
generated JSON, SVG, wheel	record outputs for reproducible submission	generate; do not hand-edit

```
# Python source: a caller outside the package
from ap_week02_path.path_geometry import interpolate_segment

points = interpolate_segment((0.0, 0.0), (3.0, 4.0), 5)
print(points.shape)
```

This absolute import has three readable parts: `ap_week02_path` is the package, `path_geometry` is the module, and `interpolate_segment` is the imported public function. It makes the function available to an outside caller; it does not launch a terminal command. The protected in-package demo uses the equivalent relative form `from .path_geometry import ...`.

```
# PowerShell: run from the repository root
python scripts/generate_artifacts.py
# writes artifacts/path_geometry_preview.svg
```

The PowerShell command launches a separate supplied script. That script imports the numerical module, consumes its returned array, and writes an SVG. In Week 2, verification means numerical contract verification: deterministic tests check shapes, endpoint preservation, numerical values, declared exceptions, and input non-mutation. A generated picture is useful execution evidence, but it does not replace those checks.

2.3 Deterministic pure-function unit tests make selected contract statements executable

The Week 2 pytest examples are deterministic unit tests of pure numerical functions: the same explicit input should produce the same return value or declared exception without files, displays, clocks, or external services. A test chooses an input whose expected relation is known, calls one public function, and checks the result or declared failure. Normal, boundary, and invalid tests serve different purposes and should all be present before a function is treated as complete.

Category	Example	Contract evidence
normal	`(0,0)` to `(3,4)`	ordinary distance is 5
boundary	identical endpoints	zero length is accepted
invalid	shape `(3,)` or non-finite coordinate	`PathInputError` is raised

```
def test_segment_length_3_4_5():
    actual = segment_length((0.0, 0.0), (3.0, 4.0))
    assert actual == pytest.approx(5.0)


def test_segment_length_accepts_zero_length():
    assert segment_length((1.5, -2.0), (1.5, -2.0)) == 0.0


def test_segment_length_rejects_nonfinite_coordinate():
    with pytest.raises(PathInputError, match="finite"):
        segment_length((0.0, np.nan), (1.0, 1.0))
```

The invalid test checks the public exception type and a stable message fragment rather than the internal helper name. This protects the published behavior while allowing the implementation to be refactored later. Students add independent tests; they do not edit the published suite to make a defect disappear.

Read the first observable mismatch before changing the code

A pytest report names the failing test, shows the assertion, prints the actual and expected values, and then identifies the source line reached by the call. Start with the first failure whose relation is understood. Form a hypothesis from the numbers, inspect the cited expression, and rerun a focused test after one change.

```
FAILED tests/test_published_contract.py::test_segment_length_3_4_5

E       assert 7.0 == 5.0 ± 5.0e-12
E         comparison failed
E         Obtained: 7.0
E         Expected: 5.0 ± 5.0e-12
```

The obtained value 7 suggests that absolute coordinate differences may have been added. It does not by itself prove the exact source defect. The traceback and implementation must still be inspected. Repairing the first mismatch is more informative than editing several TODOs and rerunning the entire suite blindly.

Floating-point comparison states an explicit numerical policy

Many decimal values cannot be represented exactly in binary floating-point. `np.isclose` compares scalar or elementwise results, while `np.allclose` reduces an array comparison to one Boolean. The tolerance must be small enough for the scale and operations in the contract; it is not permission for an arbitrary mismatch.

```
assert np.isclose(actual_length, expected_length,
                  rtol=1e-12, atol=1e-12)
assert np.allclose(actual_points, expected_points,
                   rtol=1e-12, atol=1e-12)
```

2.4 NumPy makes point, waypoint, and batch axes explicit

A list of tuples is readable, but a numerical package benefits from a homogeneous array with a declared shape. The number of dimensions, reported by `ndim`, increases as one point becomes one path and then a batch, while the last axis continues to mean x and y.

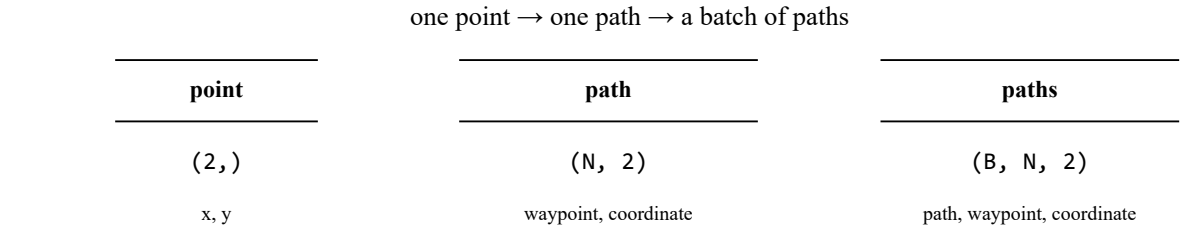

Figure 2.2. Shape is part of the data contract, not only a storage detail.

```
path = np.asarray(
    [(0.0, 0.0), (3.0, 0.0), (3.0, 4.0)],
    dtype=np.float64,
)

print(path.ndim)
print(path.shape)
print(path[1, 0])
```

```
2
(3, 2)
3.0
```

Representation	Shape	Axis meaning
point	`(2,)`	axis 0: x and y
path	`(N,2)`	axis 0: waypoint; axis 1: coordinate
batch	`(B,N,2)`	axis 0: path; axis 1: waypoint; axis 2: coordinate

A slice may share memory with caller-owned input

Basic NumPy slicing commonly creates a view. The view has its own array object but refers to the same underlying data. An in-place assignment through the view can therefore change the caller's array. Public numerical functions in this assignment promise not to mutate their inputs.

```
path = np.array([[0.0, 0.0], [3.0, 0.0], [3.0, 4.0]])
middle_view = path[1:2]
middle_view[0, 0] = 99.0
print(path)
```

```
[[ 0.  0.]
 [99.  0.]
 [ 3.  4.]]
```

The slice `1:2` keeps the waypoint axis because it selects a length-one range. It also shares memory. An independent working array requires `copy()`, but a read-only calculation often needs no copy at all. Avoiding in-place assignment is both clearer and cheaper.

```
candidate = np.array([[0.0, 0.0], [3.0, 0.0], [3.0, 4.0]])
snapshot = candidate.copy()
_ = path_length(candidate)
assert np.array_equal(candidate, snapshot)
```

Expression	Typical result	Ownership consequence
`path[1]`	shape `(2,)`	select one waypoint
`path[1:2]`	shape `(1,2)` view	assignment may alter `path`
`path[1:2].copy()`	shape `(1,2)` copy	independent data

Broadcasting evaluates all interpolation parameters in one expression

Linear interpolation evaluates the point between a start and a goal at parameter alpha. A column of alpha values has shape `(M,1)`; the displacement has shape `(1,2)`. Broadcasting expands the dimensions of length one and returns one two-coordinate row for each alpha.

$$p(\alpha) = (1 - \alpha)\, p_{start} + \alpha\, p_{goal},\ 0 \le \alpha \le 1$$

```
start = np.asarray((0.0, 0.0), dtype=np.float64)  # (2,)
goal = np.asarray((3.0, 4.0), dtype=np.float64)   # (2,)
alpha = np.linspace(0.0, 1.0, 5)[:, None]        # (5, 1)

points = start[None, :] + alpha * (goal - start)[None, :]
print(points.shape)
print(points)
```

```
(5, 2)
[[0.   0.  ]
 [0.75 1.  ]
 [1.5  2.  ]
 [2.25 3.  ]
 [3.   4.  ]]
```

The first row is the start and the last row is the goal. The output shape `(5,2)` is a contract statement: five returned points, each with x and y. A smooth visual line alone cannot reveal an omitted endpoint or reversed coordinate order, so both rows are tested numerically.

The sample count includes both endpoints

The public parameter `num_samples` means the total returned rows, including both endpoints. It accepts a Python or NumPy integer M at least two. Boolean is rejected because `True` does not communicate a meaningful sampling request.

M	Returned points	Contract result
1	start only or goal only	invalid: cannot preserve both endpoints
2	start, goal	minimum valid result
5	start, three interior points, goal	valid five-row result

```
if (
    isinstance(num_samples, (bool, np.bool_))
    or not isinstance(num_samples, (int, np.integer))
):
    raise PathInputError("num_samples must be an integer at least 2")
if int(num_samples) < 2:
    raise PathInputError("num_samples must be an integer at least 2")
```

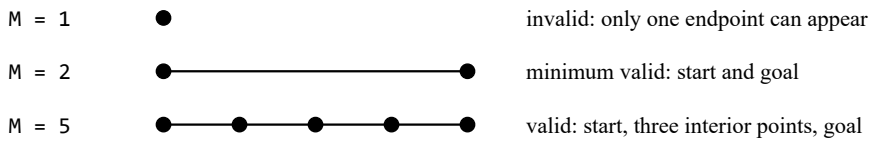

Figure 2.3. The sample count is the total number of returned points, including both endpoints.

Boundary tests check M = 2 and reject M = 1, non-integers, and Boolean. Endpoint equality is checked separately so that the right shape cannot hide wrong points.

2.5 A polyline length sums consecutive segment lengths

A path length depends on every ordered waypoint. It is not generally the direct distance from the first point to the last. The scalar loop remains the most inspectable reference because each pair can be traced using the Python review from the beginning of the chapter.

$$P = [p_0, p_1, \ldots, p_{N-1}], N \geq 2$$
$$L(P) = \Sigma_{i=0}^{N-2} ||p_{i+1} - p_i||_2$$

```
def path_length_scalar_reference(path_xy) -> float:
    path = np.asarray(path_xy, dtype=np.float64)
    total = 0.0
    for p0, p1 in zip(path[:-1], path[1:]):
        total += segment_length(p0, p1)
    return total

path = np.array([[0.0, 0.0], [3.0, 0.0], [3.0, 4.0]])
print(segment_length(path[0], path[-1]))
print(path_length_scalar_reference(path))
```

```
5.0
7.0
```

The direct distance is 5.0. The polyline travels three units to the corner and four units to the goal, so its length is 7.0. The intermediate waypoint changes the second question while leaving the endpoints unchanged. This distinction later lets a planner compare different feasible routes between the same start and goal.

The public function must accept only a finite array of shape `(N,2)` with N at least two. Repeated waypoints are valid and contribute a zero-length segment.

Vectorization preserves the same consecutive-pair calculation

NumPy can create every segment displacement at once. The two slices keep the waypoint order and both have shape `(N-1,2)`. Reduction along axis 1 combines x and y within each segment; the final sum combines the segment lengths along the path.

```
# kernel after public input has been validated as finite shape (N,2)
path = np.asarray(path_xy, dtype=np.float64)
displacement = path[1:, :] - path[:-1, :]      # (N-1, 2)
squared = displacement * displacement           # (N-1, 2)
segment_lengths = np.sqrt(np.sum(squared, axis=1))
result = float(np.sum(segment_lengths))
```

Expression	Shape	Meaning
`path[1:] - path[:-1]`	`(N-1,2)`	one displacement per segment
`sum(..., axis=1)`	`(N-1,)`	squared length of each segment
`sqrt(...)`	`(N-1,)`	Euclidean segment lengths
final `sum`	scalar	total polyline length

Using axis 0 in the squared-sum step would combine different segments by coordinate and return two values. Shape tracing is therefore part of the correctness argument. The vectorized result is compared against the scalar loop rather than being trusted merely because it is shorter or faster.

```
actual = path_length(path)
expected = path_length_scalar_reference(path)
assert np.isclose(actual, expected, rtol=1e-12, atol=1e-12)
```

A batch adds one axis without changing one-path meaning

The public `path_lengths` contract accepts only finite input of shape `(B,N,2)` with $B \geq 1$ paths and $N \geq 2$ waypoints per path. It raises `PathInputError` for an empty batch ($B = 0$), an empty path axis ($N = 0$), or a one-waypoint path axis ($N = 1$). For valid input, the implementation reduces coordinate differences into segment lengths and then into one length per path while preserving the batch axis.

```
displacement = paths[:, 1:, :] - paths[:, :-1, :]  # (B,N-1,2)
segment_lengths = np.sqrt(np.sum(displacement**2, axis=2))
lengths = np.sum(segment_lengths, axis=1)           # (B,)

batch_expected = np.array([path_length(row) for row in paths])
assert np.allclose(lengths, batch_expected)

snapshot = paths.copy()
_ = path_lengths(paths)
assert np.array_equal(paths, snapshot)
```

Consistency check	Expected relation	Defect exposed
scalar-batch	entry i equals `path_length(paths[i])`	wrong axis or row alignment
permutation	row permutation causes the same output permutation	hidden row-index dependence
translation	one offset applied to all points preserves length	absolute-position leakage
no mutation	input equals its snapshot after the call	in-place view assignment

The comprehension in `batch_expected` belongs to the test oracle, not to `path_lengths`. The student batch implementation uses NumPy operations over all paths; the scalar calls provide an independently structured comparison.

2.6 Guided Lab A: complete and test the two segment functions

Begin from the unmodified starter repository and work from its root. The protected baseline command should report exactly `42 failed, 6 passed`. That baseline confirms that the environment and protected tests run while the four student functions remain incomplete. Copy the summary and first failing pytest test node ID into section 0 of `artifacts/engineering_note.md` before editing the TODO file.

```
python scripts/verify_environment.py
python scripts/run_baseline.py

# after implementing segment_length
python -m pytest -q tests/test_published_contract.py -k segment_length -x

# after implementing interpolate_segment
python -m pytest -q tests/test_published_contract.py -k interpolation -x
```

Step	Student action	Checkpoint
A1	record the baseline summary and first failing test ID	`42 failed, 6 passed`
A2	complete validation and `segment_length`	3-4-5 case returns `5.0`; zero length accepted
A3	complete `interpolate_segment`	five-sample result has shape `(5,2)`
A4	run boundary and invalid tests	both endpoints kept; M=1, Boolean, wrong shape rejected
A5	inspect `git diff` and commit	only permitted student files changed

Treat the passing focused tests for the two segment functions as the first implementation checkpoint. Do not try to finish every TODO at once. Read each failure, change one dependency, and rerun the smallest relevant test selection before the full suite.

Guided Lab B: preserve the scalar meaning in the path batch

Continue only after Lab A is stable. Implement `path_length` first, verify the direct distance 5.0 versus the corner path 7.0, and then implement `path_lengths` with NumPy operations over shape `(B,N,2)`. Passing all four public functions completes the core implementation checkpoint; the remaining steps assemble the independent evidence package.

```
$contract = "tests/test_published_contract.py"
python -m pytest -q $contract -k "path_length and not path_lengths" -x
python -m pytest -q $contract -k path_lengths -x
python -m pytest -q

python -m ap_week02_path.demo
python scripts/generate_artifacts.py
python -m build --wheel --no-isolation
python scripts/check_submission.py
```

Core implementation checkpoint	Evidence-package completion
all four public functions implemented	at least six independent top-level student tests
48 published tests pass	numerical artifacts and animated SVG generated
scalar-batch and no-mutation checks pass	all engineering-note prompts completed
focused failures understood	wheel built and clean-installed; Git revision pushed; LMS form submitted

The supplied visualization must begin at the first returned row, pass through every interpolated row, and finish at the last. It is execution evidence, not a substitute for numerical assertions. The final checker reruns the evidence, checks the note, compares generated artifacts with current outputs, and clean-installs the wheel. Assignment 2 gives the complete GitHub and LMS submission procedure.

2.7 Exercises and the transition to named domain objects

A visible path becomes trustworthy numerical evidence only after its representation, function contract, validation, scalar reference, vectorized batch calculation, and independent consistency checks agree. The chapter used tuples, lists, functions, and arrays so that each transformation could be traced directly. The following exercises consolidate that procedural and numerical foundation before Week 3 introduces named objects with explicit responsibilities.

Exercise 1

Trace the names and values produced by the page 2 example. Explain why replacing `return value**0.5` with `print(value**0.5)` changes what the caller can test.

Exercise 2

Predict the exact shape and rows of `interpolate_segment((1,2), (5,6), 3)`. Identify which two rows prove that the endpoint contract is satisfied.

Exercise 3

Construct two three-waypoint paths with the same endpoints but different lengths. Calculate both scalar results and explain why endpoint distance alone cannot rank the paths.

Exercise 4

For input shape `(B,N,2)`, trace the shape after consecutive subtraction, coordinate reduction, and waypoint reduction. Explain why reducing along axis 0 would destroy the one-output-per-path contract.

Exercise 5

Design one consistency test not based on the fixed 3-4-5 example. State the relation, the input transformation, and the implementation defect the test is intended to expose.

> Week 3 bridge. Point, Path, PlanningProblem, and PlanResult will provide named domain objects. The tested Week 2 numerical functions remain reusable foundations rather than being rewritten inside each class.